

TALLER 2

DOCENTE:

FREDDY ALEXANDER ORJUELA

PRESENTADO POR:

EDUAR AMAYA QUENGUAN

ISMAEL LARA GOMEZ

HERNY DONOSO PARRA

UNIVERSIDAD ECCI

SISTEMAS AVANZADOS DE PRODUCCION

FACULTAD DE INGENIERIA INDUSTRIAL

BOGOTÁ D.C. 2026

INTRODUCCION

A lo largo del desarrollo de la materia de Sistemas Avanzados de Producción, se ha podido entender cómo la tecnología y el análisis de datos juegan un papel cada vez más importante en la solución de problemas reales en la ingeniería. Más allá de ver modelos de manera teórica, el enfoque ha estado en comprender cómo estos pueden aplicarse para tomar decisiones más acertadas, reducir la incertidumbre y optimizar procesos en distintos contextos productivos.

En este sentido, el taller permite integrar varios de los conceptos que se han trabajado durante el curso, especialmente en lo relacionado con el uso de modelos matemáticos, técnicas de optimización y herramientas de aprendizaje automático. Desde la aplicación de la regresión Ridge para controlar problemas como la multicolinealidad, hasta el análisis del comportamiento del descenso de gradiente y el uso de redes neuronales, se evidencia cómo la combinación entre matemáticas y computación permite abordar problemas complejos de una manera más estructurada.

Lo más relevante de este tipo de ejercicios es que no se trata únicamente de obtener un resultado numérico, sino de interpretar lo que hay detrás de cada modelo, entender sus limitaciones y reconocer su impacto en la toma de decisiones. De esta forma, el componente analítico se vuelve fundamental, ya que permite pasar de simplemente aplicar fórmulas a realmente comprender el comportamiento de los sistemas y su relación con escenarios reales dentro de la ingeniería industrial.

Taller 2: Redes Neuronales

Ingeniería Industrial - Sistemas Avanzados de Producción

Fase 1: Desarrollo Analítico: Regularización Ridge.

En escenarios de alta dimensionalidad o cuando existe una fuerte correlación entre los predictores (como podría ocurrir entre la inversión en *TV* y *Radio*), el estimador de Mínimos Cuadrados Ordinarios (OLS) tiende a presentar una varianza excesiva. Para mitigar esto, se utiliza la Regresión Ridge, la cual añade de una penalización a la magnitud de los coeficientes. La función de costo a minimizar es la siguiente:

$$J(\beta) = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (1)$$

Tarea de Derivación Matemática:

Utilizando álgebra matricial, demuestre paso a paso como se llega a la solución cerrada del estimador Ridge. Para ello, siga esta ruta lógica:

1. **Representación Vectorial:** Expresé la función de costo $J(\beta)$ en términos de la matriz de diseño X , el vector de respuestas Y , el vector de parámetros β y el parámetro de regularización λ . *Pista: Recuerde que $\sum \beta^2$ se puede escribir como el producto interno del vector consigo mismo.*
2. **Expansión de la Forma Cuadrática:** Desarrolle el producto $(Y - X\beta)^T (Y - X\beta)$ y combine los términos semejantes con el término de penalización $\lambda \beta^T \beta$.
3. **Condición de Primer Orden:** Calcule el gradiente de la función de costo respecto al vector β (derivada matricial $\frac{\partial J(\beta)}{\partial \beta}$) e igualelo al vector nulo.
4. **Solución Cerrada:** Despeje el vector β para demostrar que el estimador Ridge está dado por:

$$\hat{\beta}_{Ridge} = (X^T X + \lambda I)^{-1} X^T Y$$

donde I es la matriz identidad de dimensiones adecuadas.

Fase 1

Demostración paso a paso a Solución Cerrada del
Estimador Ridge

$$1.1 \quad J(\beta) = \left(\sum_{i=1}^n y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

$$J(\beta) = (Y - X\beta)^t (Y - X\beta) + \lambda \beta^t \beta$$

$$Y = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \\ y_8 \\ y_9 \\ y_{10} \end{pmatrix} \quad X = \begin{pmatrix} 1 & x_{11} & x_{12} & x_{13} \\ 1 & x_{21} & x_{22} & x_{23} \\ 1 & x_{31} & x_{32} & x_{33} \\ 1 & x_{41} & x_{42} & x_{43} \\ 1 & x_{51} & x_{52} & x_{53} \\ 1 & x_{61} & x_{62} & x_{63} \\ 1 & x_{71} & x_{72} & x_{73} \\ 1 & x_{81} & x_{82} & x_{83} \\ 1 & x_{91} & x_{92} & x_{93} \\ 1 & x_{10,1} & x_{10,2} & x_{10,3} \end{pmatrix} \quad \beta = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \beta_3 \\ \beta_4 \end{pmatrix}$$

$$\beta^t \beta = \beta_1^2 + \beta_2^2 + \beta_3^2$$

$$\sum_{j=1}^p \beta_j^2 = \beta^t \beta$$

1.2

Expansión de Forma Cuadrática

$$\text{Producto} = (Y - X\beta)^t (Y - X\beta)$$

$$(Y - X\beta)^t (Y - X\beta) = Y^t Y - Y^t X \beta - (X\beta)^t Y + (X\beta)^t (X\beta)$$

$$J(\beta) = Y^t Y - Y^t X \beta - (X\beta)^t Y + (X\beta)^t (X\beta) + \lambda \beta^t \beta$$

7.3

Condición de primer orden

Derivada respecto a β : $\frac{\partial J(\beta)}{\partial \beta}$

$$\frac{\partial}{\partial \beta} (Y^T Y) = 0$$

$$\frac{\partial}{\partial \beta} (-2\beta^T X^T Y) = -2X^T Y$$

$$\frac{\partial}{\partial \beta} (\beta^T X^T X \beta) = 2X^T X \beta$$

$$\frac{\partial}{\partial \beta} (\lambda \beta^T \beta) = 2\lambda \beta$$

Gradiente: $\frac{\partial J(\beta)}{\partial \beta} = -2X^T Y + 2X^T X \beta + 2\lambda \beta$

Iguando a Vector nulo: $-2X^T Y + 2X^T X \beta + 2\lambda \beta = 0$ para función de costo

7.4

$$-2X^T Y + 2X^T X \beta + 2\lambda \beta = 0$$

$$= -X^T Y + X^T X \beta + \lambda \beta = 0$$

$$= X^T X \beta + \lambda \beta = X^T Y$$

$$= (X^T X + \lambda I) \beta = X^T Y$$

Despejando $\hat{\beta}_{\text{Ridge}} = (X^T X + \lambda I)^{-1} X^T Y$

Preguntas Fase 1

- **Sobre la Invertibilidad:** ¿Por qué la suma de λI garantiza que la matriz resultante sea invertible incluso si $X^T X$ es singular (no invertible)?

RTA: En la regresión lineal tradicional, la matriz puede no ser invertible cuando existe multicolinealidad entre las variables independientes, es decir, cuando algunas variables están altamente correlacionadas entre sí. Esto genera problemas para encontrar una solución única del modelo.

Al introducir el término de regularización, se están sumando valores positivos en la diagonal de la matriz. Esto modifica sus propiedades matemáticas, haciendo que la matriz resultante sea definida positiva y, por lo tanto, siempre invertible.

En términos prácticos, este ajuste estabiliza el modelo y evita problemas numéricos, permitiendo obtener una solución única incluso cuando los datos presentan alta correlación.

- **Efecto del Parámetro λ :** ¿Qué sucede con el modelo y sus coeficientes cuando $\lambda \rightarrow 0$? ¿Y qué ocurre cuando $\lambda \rightarrow \infty$? Explique en términos de la complejidad del modelo (Sesgo vs. Varianza).

RTA: El parámetro λ controla el nivel de penalización sobre los coeficientes del modelo, influyendo directamente en su complejidad. Cuando $\lambda \rightarrow 0$ el término de regularización prácticamente desaparece y el modelo se comporta como una regresión lineal ordinaria (OLS). En este caso, el modelo tiende a ajustarse mucho a los datos, lo que puede reducir el sesgo, pero aumentar la varianza, haciéndolo más sensible a cambios en los datos.

Cuando $\lambda \rightarrow \infty$, la penalización sobre los coeficientes es muy fuerte, lo que hace que estos tiendan a valores cercanos a cero. Esto simplifica el modelo de manera considerable, aumentando el sesgo, pero reduciendo la varianza.

Fase 2: Optimización Numérica y Descenso de Gradiente

En esta fase, exploraremos el “motor” que permite entrenar modelos complejos como las Redes Neuronales: el algoritmo de Descenso de Gradiente. El objetivo es entender como la configuración de los hiperparámetros determina si un modelo logra converger al mínimo global de la función de pérdida o si falla en el intento. Para el desarrollo de esta sección utilice el notebook disponible en el siguiente enlace: [GitHub: Semana 6 - Descenso de Gradiente](#).

Ubique el ultimo bloque de código donde se configura la simulación:

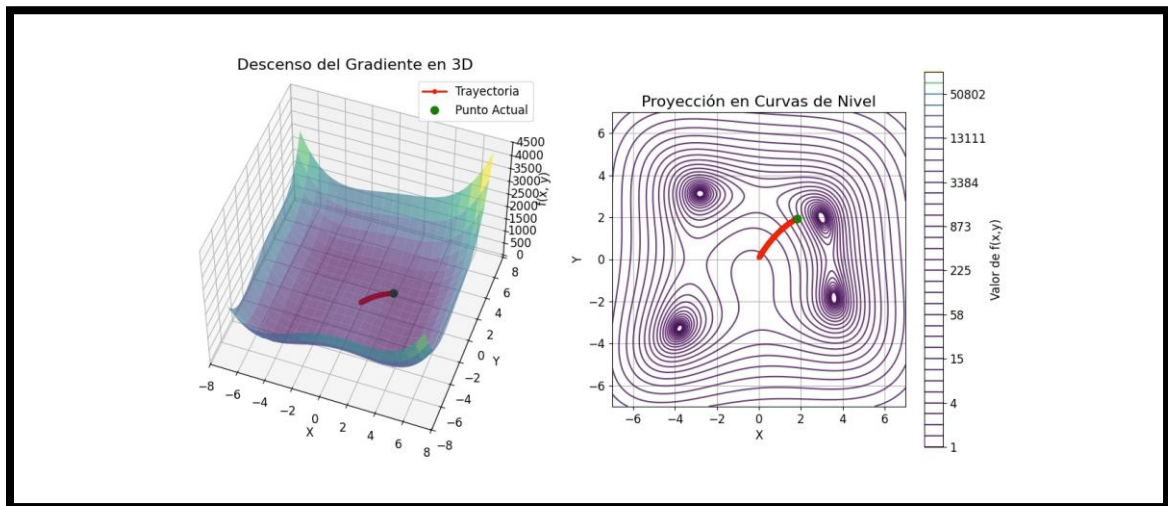
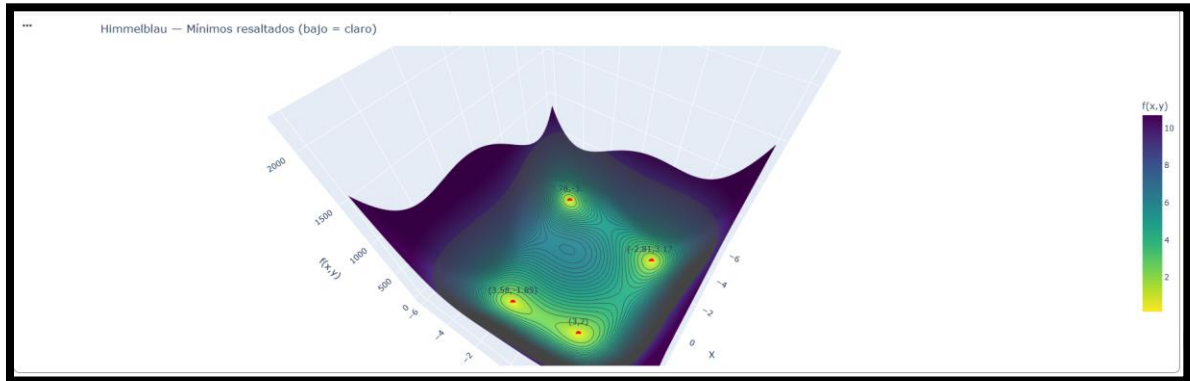
```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
start_point = [0.0, 0.1]      # Punto de inicio
learning_rate = 0.0001       # Tasa de aprendizaje (eta)
num_iterations = 1000         # Número de pasos
```

Tareas de Experimentación

Utilizando la visualización de la superficie 3D y las curvas de nivel generadas por la malla (`np.meshgrid`), realice las siguientes variaciones y documente los cambios observados en la trayectoria del algoritmo:

1. Impacto de la Tasa de Aprendizaje (η):

- Mantenga el `start_point` y las `num_iterations` constantes.



- Incremente el learning rate a un valor significativamente más alto (ej. 0.5 o 1.0). ¿Qué sucede con la trayectoria? ¿Logra llegar al centro de las curvas de nivel o comienza a oscilar/divergir?

RTA: Se incrementó el learning rate a 0.5 como se observa en la gráfica la trayectoria se vuelve inestable y no converge y no se acerca al mínimo.

```

point = np.array(start_pos, dtype=float)

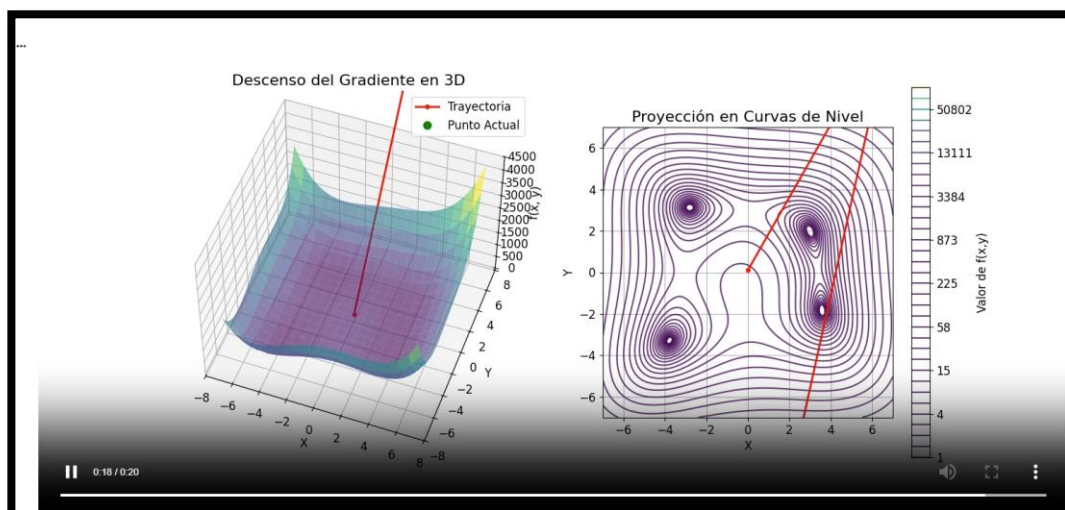
for i in range(num_iterations):
    z = f(point[0], point[1])
    path.append([point[0], point[1], z])

    # Calcula el gradiente
    grad = grad_f(point[0], point[1])

    # Actualiza la posición del punto
    point = point - learning_rate * grad

return np.array(path)

# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [0.0, 0.1]
learning_rate = 0.5      # Tasa de aprendizaje
num_iterations = 1000    # Número de pasos
    
```

- Reduzca el learning rate a un valor extremadamente pequeño (ej. 10^{-7}). Observe el resultado tras 1000 iteraciones. ¿Es suficiente este tiempo de cómputo para alcanzar el mínimo?

RTA: Se observa que el tiempo de cómputo de 1000 iteraciones no es suficiente para alcanzar el mínimo cuando se utiliza una tasa de aprendizaje extremadamente pequeña. Esto se debe a que los pasos que realiza el algoritmo en cada iteración son muy reducidos, lo que provoca un avance muy lento hacia el mínimo como se observa ni se mueve del punto inicial.

```

Semana6.ipynb  Guardar en GitHub para conservar los cambios
Archivo  Editar  Ver  Insertar  Entorno de ejecución  Herramientas  Ayuda

Comandos  + Código  + Texto  ▶ Ejecutar todas  Copiar en Drive

[6] 5 min
# Calcula el gradiente
grad = grad_f(point[0], point[1])

# Actualiza la posición del punto
point = point - learning_rate * grad

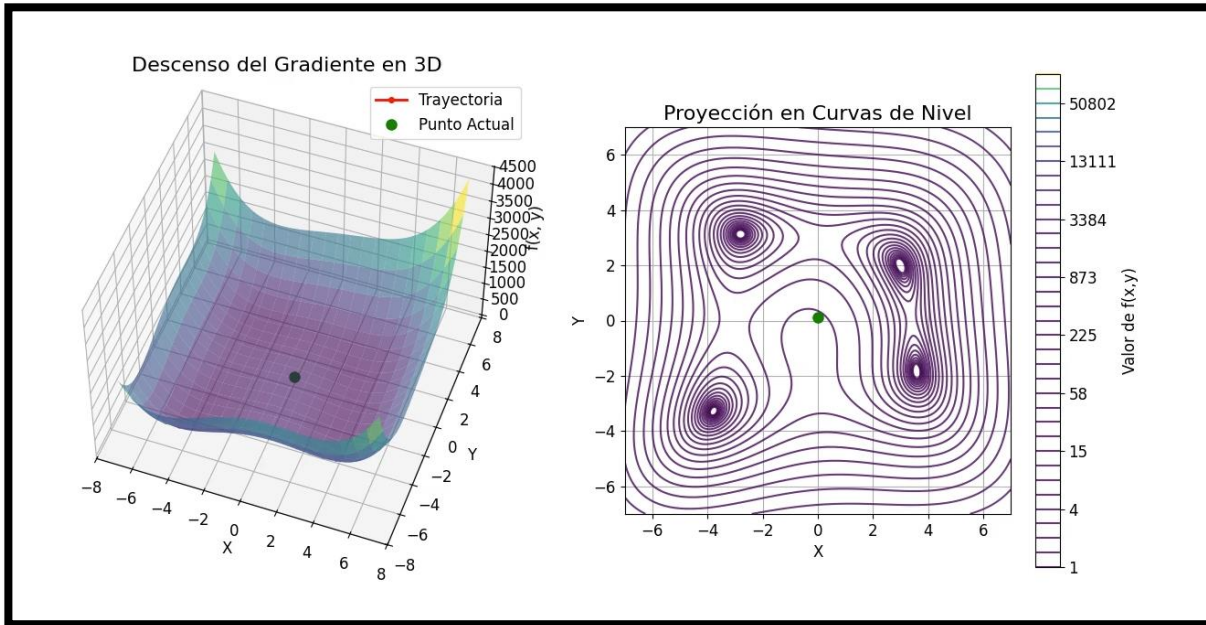
return np.array(path)

# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [0.0, 0.1]
learning_rate = 0.0000001 # Tasa de aprendizaje
num_iterations = 1000 # Número de pasos

# Ejecutar el algoritmo
path = gradient_descent(start_point, learning_rate, num_iterations)

# --- PREPARACIÓN DE LA VISUALIZACIÓN ---
# Crear la malla de puntos para la superficie 3D y las curvas de nivel
x = np.linspace(-7, 7, 200)
y = np.linspace(-7, 7, 200)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

```

2. Dependencia del Punto Inicial:

- Modifique el `start_point` a coordenadas alejadas del origen, por ejemplo `[6.0, -6.0]` o `[-5.0, 5.0]`.

```
[?] 5 min
✓ min

# Calcula el gradiente
grad = grad_f(point[0], point[1])

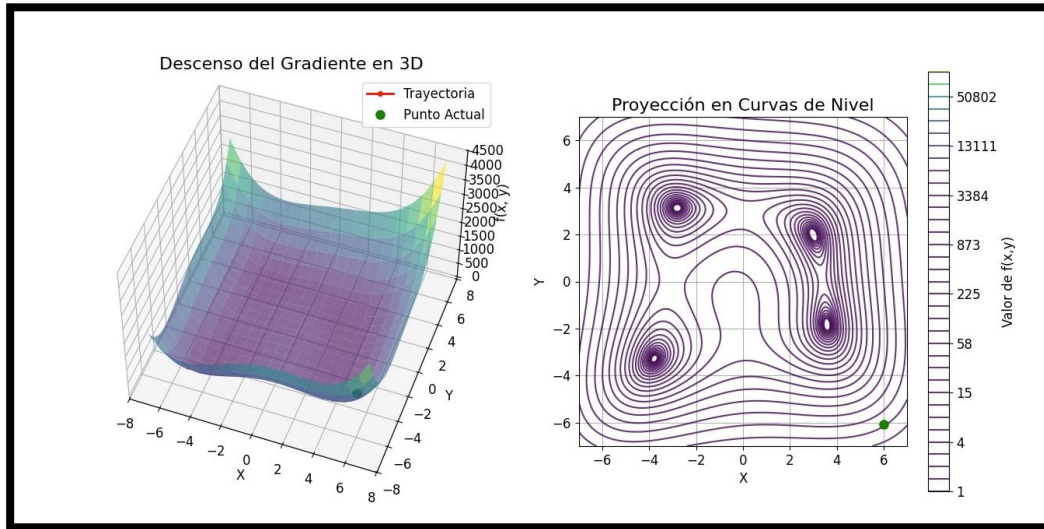
# Actualiza la posición del punto
point = point - learning_rate * grad

return np.array(path)

# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [6.0, -6.1]
learning_rate = 0.0000001 # Tasa de aprendizaje
num_iterations = 1000 # Número de pasos

# Ejecutar el algoritmo
path = gradient_descent(start_point, learning_rate, num_iterations)

# --- PREPARACIÓN DE LA VISUALIZACIÓN ---
# Crear la malla de puntos para la superficie 3D y las curvas de nivel
x = np.linspace(-7, 7, 200)
y = np.linspace(-7, 7, 200)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)
```



```
[8] 5 min
▶ for i in range(num_iterations):
    z = f(point[0], point[1])
    path.append([point[0], point[1], z])

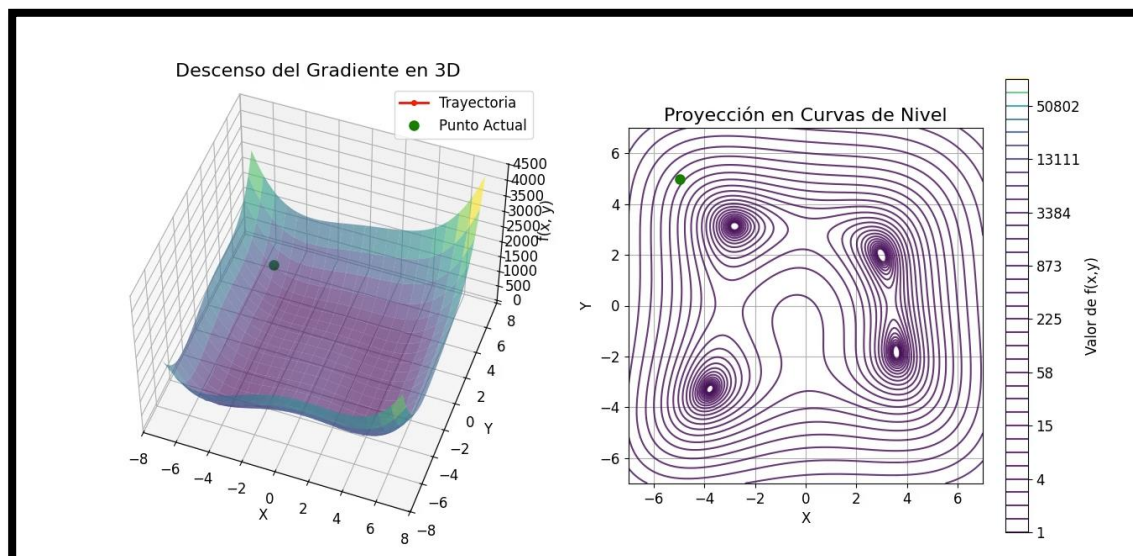
    # Calcula el gradiente
    grad = grad_f(point[0], point[1])

    # Actualiza la posición del punto
    point = point - learning_rate * grad

return np.array(path)

# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [-5.0, 5.0]
learning_rate = 0.0000001 # Tasa de aprendizaje
num_iterations = 10000 # Número de pasos

# Ejecutar el algoritmo
path = gradient_descent(start_point, learning_rate, num_iterations)
```



Preguntas Fase 2

A partir de sus observaciones en la simulación, responda de manera crítica:

- **El fenómeno de “Overshooting”:** Explique físicamente que ocurre cuando el paso ($\eta \cdot \nabla f$) es más grande que la distancia al mínimo. ¿Como se visualiza esto en el grafico de curvas de nivel?

RTA: Este fenómeno ocurre cuando el tamaño del paso del algoritmo, determinado por el producto entre la tasa de aprendizaje (η) y el gradiente (∇f), es mayor que la distancia al mínimo. En este caso, el algoritmo no se detiene cerca del mínimo, sino que lo sobrepasa en cada iteración. Físicamente, esto se puede interpretar como un sistema que tiene demasiada “inercia”, lo que le impide estabilizarse en el punto de equilibrio. En el gráfico se visualiza como una trayectoria que salta de un lado a otro del mínimo, cruzando repetidamente las curvas sin acercarse progresivamente al centro como se observó al principio en los primeros ejercicios

- **Eficiencia Computacional:** Si el algoritmo se detiene en la iteración 1000 pero aún se encuentra lejos del mínimo, ¿Que estrategia es mejor: aumentar el número de iteraciones o ajustar la tasa de aprendizaje? Justifique su respuesta.

RTA: Lo mejor es aumentar la tasa de aprendizaje como observamos cuando esta es muy baja, el algoritmo avanzará demasiado lento, por lo cual aumentar las iteraciones solo incrementará el tiempo de cómputo sin garantizar una mejora significativa en la convergencia. Por el contrario, cuando ajustamos la tasa de aprendizaje permite encontrar un equilibrio adecuado entre velocidad y estabilidad, logrando que el algoritmo se acerque al mínimo de manera más eficiente.

Fase 3: Redes Neuronales (MLP) y Ajuste de Hiperparámetros

En esta fase, implementaremos un **Perceptrón Multicapa (MLP)**, una arquitectura de red neuronal densa capaz de capturar relaciones no lineales complejas entre la inversión publicitaria y las ventas. A diferencia de la regresión lineal, aquí el modelo aprende representaciones abstractas en sus capas ocultas.

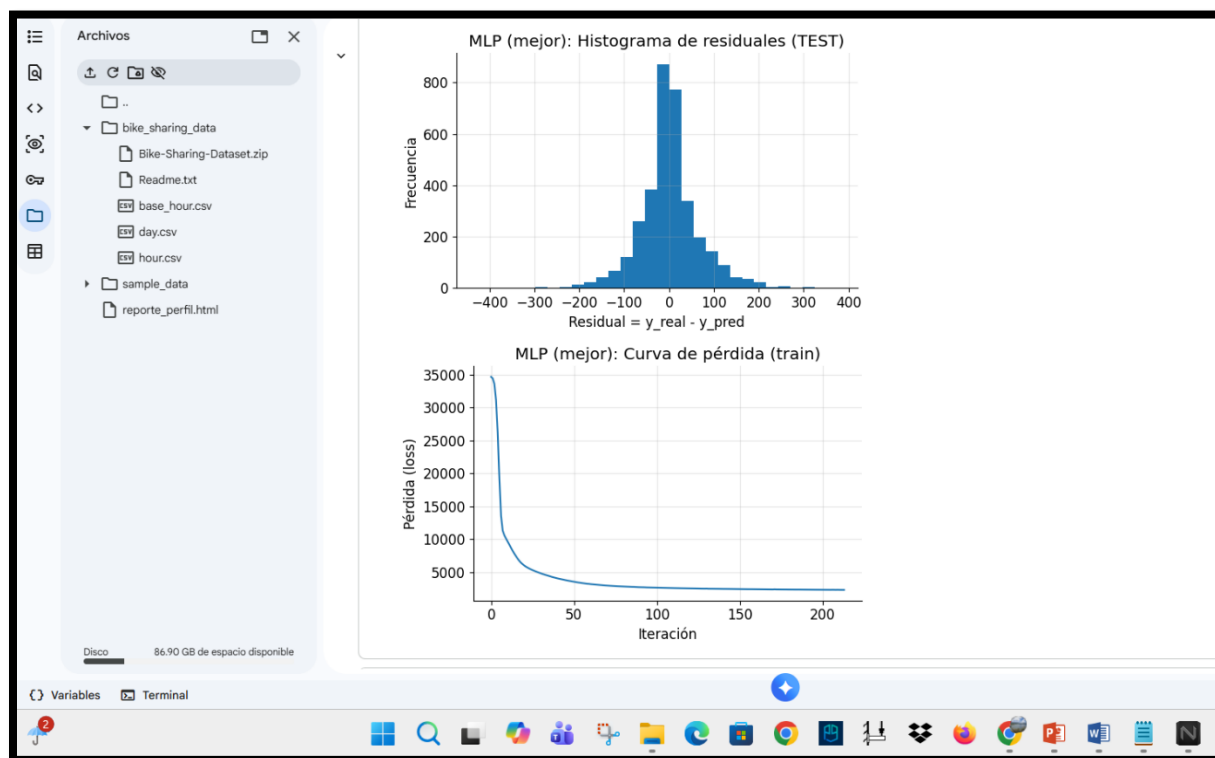
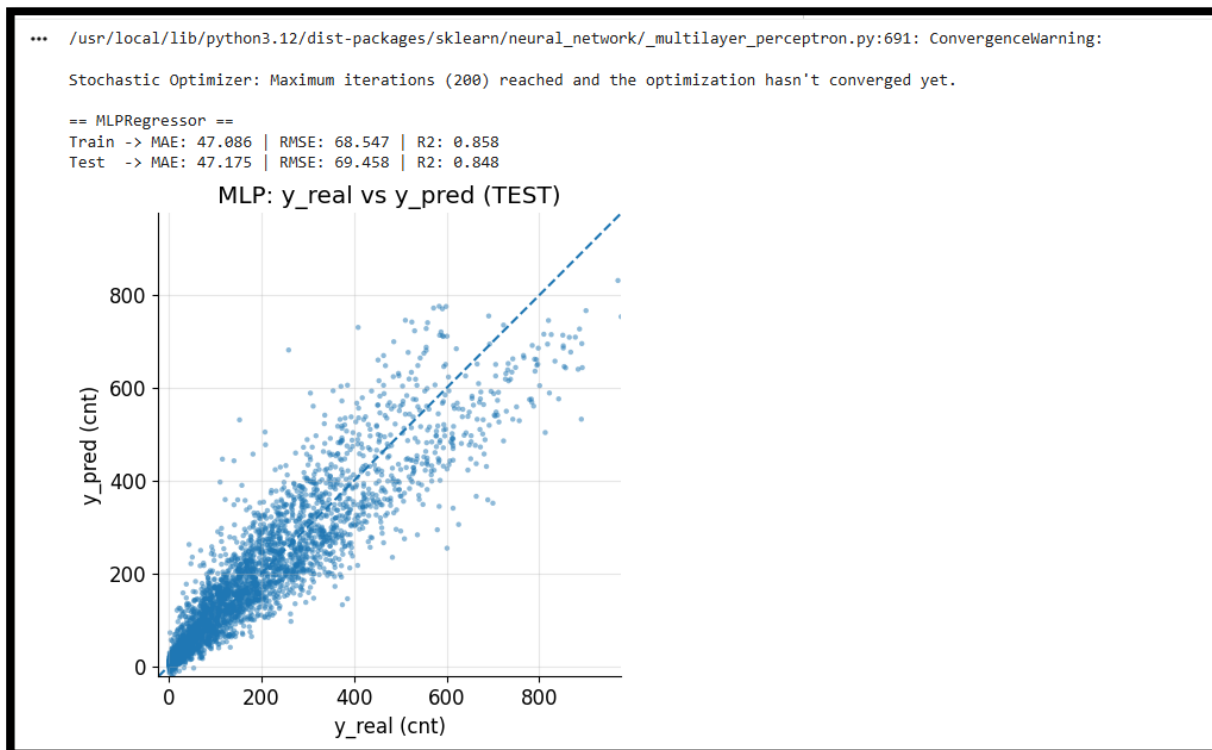
Parte A: Evaluación de Métricas de Regresión

Ejecute el modelo `MLPRegressor` inicial con la configuración base (`hidden layer sizes= (16, 8, 4)`). Observe los resultados de las métricas **MAE**, **RMSE** y R^2 tanto para el conjunto de entrenamiento como para el de prueba (test).

== `MLPRegressor` ==

Train -> MAE: 47.086 | RMSE: 68.547 | R2: 0.858

Test -> MAE: 47.175 | RMSE: 69.458 | R2: 0.848



Preguntas Fase 3A:

- **Diferenciación de Métricas:** Explique la diferencia técnica entre el Error Absoluto Medio (MAE) y la Raíz del Error Cuadrático Medio (RMSE). ¿Cuál de las dos penaliza con mayor

severidad los errores grandes (*outliers*) en las ventas y por qué?

RTA: El MAE muestra el error promedio “real” que comete el modelo, es decir, cuánto se equivoca en unidades directas sin exagerar los errores. En cambio, el RMSE castiga más los errores grandes porque los eleva al cuadrado antes de promediarlos. Por eso, si hay predicciones muy malas, el RMSE sube bastante más que el MAE. En otras palabras, el RMSE es más “estricto” con los errores grandes, mientras que el MAE es más equilibrado

- **Diagnóstico de Desempeño:** Si el R^2 en el conjunto de entrenamiento es significativamente mayor que en el de prueba, ¿Qué fenómeno está ocurriendo? Proponga una solución basada en lo aprendido en la Fase 1 del taller.

RTA: El modelo presenta valores de R^2 muy similares entre entrenamiento (0.859) y prueba (0.852), lo que indica que no hay sobreajuste significativo. Esto significa que el modelo tiene una buena capacidad de generalización, ya que mantiene un desempeño consistente en datos no vistos.

- **Interpretación de Residuos:** Al observar la gráfica de *y real vs y pred*, ¿se identifica algún patrón de error sistemático (ej. subestimación en ventas altas)?

RTA: Se observa que la mayoría de los puntos siguen una tendencia cercana a la línea diagonal, lo que indica que el modelo está prediciendo de forma bastante consistente. Sin embargo, a medida que aumentan los valores reales (ventas más altas), los puntos comienzan a dispersarse más y muchos quedan por debajo de la línea ideal. Esto sugiere que el modelo tiende a subestimar ligeramente las ventas altas.

1.1.1. Parte B: Optimización con GridSearchCV

La eficacia de una red neuronal depende críticamente de sus hiperparametros. A continuación, se presenta una guía de referencia completa para orientar su proceso de optimización:

Guía de Referencia: Hiperparametros del MLPRegressor

Tarea de Experimentación:

Utilizando la Tabla 2 como guía, modifique la parrilla de parámetros (param grid) en el notebook y ejecute la búsqueda sistemática (*GridSearch*). Se sugiere variar estratégicamente los siguientes elementos:

1. **Arquitectura (hidden layer sizes):** Compare una red “profunda” (más capas, ej. (32,16,8,4)) contra una “ancha” (una sola capa con muchas neuronas, ej. (64,)).
2. **Regularización (alpha):** Conecte esto con la Fase 1. Pruebe valores como [1e-5, 1e-2, 1.0]. ¿Como afecta un α muy elevado a la capacidad de aprendizaje?
3. **Tasa de Aprendizaje (learning rate init):** Conecte esto con la Fase 2. Analice como influye en el número de iteraciones necesarias para que el *Early Stopping* se active.

Nota sobre complejidad combinatoria: La grilla base tiene $4 \times 3 \times 3 = 36$ combinaciones $\times 3$ *folds* = 108 ajustes. Si añade activation y batch size, supera fácilmente las 500 combinaciones.

En ese caso, considere RandomizedSearchCV con n_iter=50.

Resultado de la búsqueda (CV) ==

Mejores hiperparámetros: {'alpha': 0.01, 'hidden_layer_sizes': (32, 16, 8, 4), 'learning_rate_init': 0.001}

RMSE CV (aprox): 70.9614937560113

== MLP (MEJOR MODELO) ==

Train -> MAE: 42.779 | RMSE: 63.138 | R2: 0.880

Test -> MAE: 45.985 | RMSE: 68.610 | R2: 0.851

```

Ejecución Herramientas Ayuda
Autar todas
[15] 47 min
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import time

# --- Función auxiliar de RMSE ---
def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

# 1) Estimator base (con early_stopping interno)
mlp_base = MLPRegressor(
    activation="relu",
    solver="adam",
    max_iter=600,
    early_stopping=True,
    validation_fraction=0.2,
    n_iter_no_change=10,
    random_state=42
)

# 2) Parrilla de hiperparámetros
param_grid = {
    "hidden_layer_sizes": [
        (32, 16, 8, 4), # tu config actual
        (64), # menos profundo, más ancho
        (32, 16, 8), # un poco mayor capacidad
        (16, 8) # más simple
    ],
    "alpha": [1e-5, 1e-2, 1.0], # regularización l2
    "learning_rate_init": [1e-3, 5e-4, 1e-4] # tasa de aprendizaje inicial
}

# 3) GridSearchCV
grid = GridSearchCV(
    estimator=mlp_base,
    param_grid=param_grid,
    scoring="neg_root_mean_squared_error", # sklearn maximiza -> usamos neg-RMSE

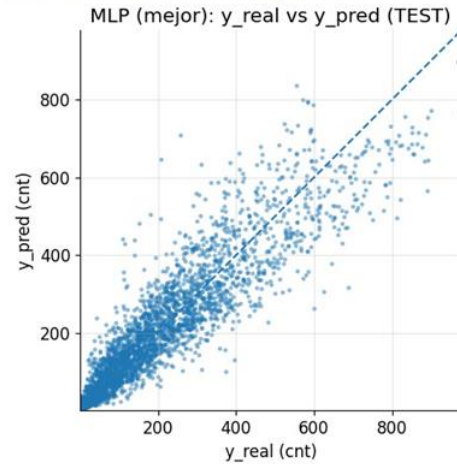
```



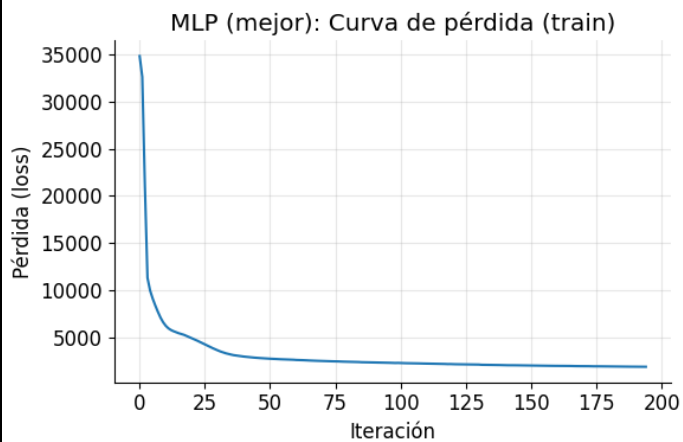
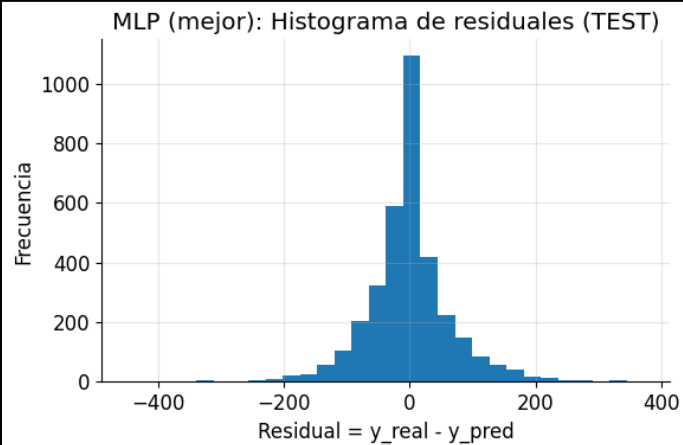
```
Ejecutando GridSearchCV (esto puede tardar un poco)...
Fitting 3 folds for each of 36 candidates, totalling 108 fits
...
Fitting 3 folds for each of 36 candidates, totalling 108 fits
GridSearchCV tomó: 1378.06 s

== Resultado de la búsqueda (CV) ==
Mejores hiperparámetros: {'alpha': 0.01, 'hidden_layer_sizes': (32, 16, 8, 4), 'learning_rate_init': 0.001}
RMSE CV (aprox): 70.9614937560113

== MLP (MEJOR MODELO) ==
Train -> MAE: 42.779 | RMSE: 63.138 | R2: 0.880
Test -> MAE: 45.985 | RMSE: 68.610 | R2: 0.851
```



MLP (mejor): Histograma de residuales (TEST)



Preguntas Fase 3B:

- **Consistencia de Parámetros:** Compare el valor óptimo de alpha encontrado por el Grid-Search con lo que analizo en la sección de Regresión Ridge. ¿Existe una relación entre la complejidad de la red y la necesidad de regularización?

RTA: El mejor valor encontrado por GridSearchCV fue: $\alpha = 0.01$ Este valor indica un nivel de regularización moderado, evitando el sobreajuste (overfitting) y no limita demasiado la capacidad de aprendizaje del modelo, la mejor arquitectura fue (32, 16, 8, 4) red profunda. Sí existe relación a mayor complejidad de la red, mayor necesidad de regularización para controlar el sobreajuste.

- **Early Stopping:** Explique como este mecanismo ayuda a optimizar el tiempo de cómputo y a prevenir el sobreajuste basado en sus resultados.

RTA: Reduce tiempo de cómputo ya que no entrena las 600 iteraciones completas y se detiene automáticamente cuando ya no mejora evitando el sobreajuste lo que impide que el modelo "memorice" los datos de entrenamiento se observa ANTES: Train RMSE: 68.547 - Test RMSE: 69.458. Con GridSearch + early stopping Train RMSE: 63.138 - Test RMSE: 68.610 al ser la Diferencia pequeña entre train y test - buena generalización Early stopping permite entrenar de forma más eficiente y mejora la capacidad de generalización del modelo.

- **Conclusión de Negocio:** Basándose en el mejor modelo encontrado, ¿cuál es el margen de error promedio que la empresa debería esperar al predecir sus ventas? ¿Es el modelo lo suficientemente robusto para decisiones presupuestales? Justifique.

RTA: El mejor modelo fue el de Grid Search: RMSE (test): 68.61, MAE (test): 45.99, R^2 : 0.851. Este modelo es suficientemente robusto para apoyar decisiones presupuestales a nivel general, ya que presenta un buen poder predictivo ($R^2 \approx 0.85$) y un error promedio aceptable. Sin embargo, para decisiones críticas o de alta precisión, se recomienda complementarlo con análisis adicionales debido al margen de error de aproximadamente 46 unidades - RMSE bajó de 69.46 a 68.61 y R^2 subió de 0.848 a 0.851.

Fase 4: Propagación hacia Adelante (Forward Propagation) Manual

En esta fase, el objetivo es comprender la mecánica interna de una Red Neuronal Multicapa. En lugar de predecir un valor continuo de ventas, clasificaremos el éxito de una campaña en tres categorías: **Bajo Impacto** (C_1), **Impacto Medio** (C_2) e **Impacto Alto** (C_3).

Arquitectura y Contexto

Considere una red neuronal con la siguiente estructura:

- **Capa de Entrada:** 3 características (x_1 : TV, x_2 : Radio, x_3 : Newspaper).
- **Capa Oculta (Capa 1):** 3 neuronas con función de activación **Sigmoide** (σ).
- **Capa de Salida (Capa 2):** 3 neuronas (una por clase) con función de activación **Softmax**.

Definición de Parámetros y Datos

Se proporcionan los siguientes pesos y sesgos (*biases*) ya entrenados para un mercado específico. El vector de entrada normalizado es $X = [0.5, 0.3, 0.2]^T$.

Capa 1 (Oculto):

$$W^{(1)} = \begin{pmatrix} 0,2 & 0,5 & -0,1 \\ 0,8 & -0,3 & 0,4 \\ -0,5 & 0,1 & 0,2 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0,1 \\ -0,2 \\ 0,0 \end{pmatrix}$$

Capa 2 (Salida):

$$W^{(2)} = \begin{pmatrix} 0,5 & -0,4 & 0,1 \\ 0,2 & 0,8 & -0,5 \\ -0,1 & 0,3 & 0,7 \end{pmatrix}, \quad b^{(2)} = \begin{pmatrix} -0,1 \\ 0,2 \\ 0,1 \end{pmatrix}$$

Tarea de Cálculo Manual

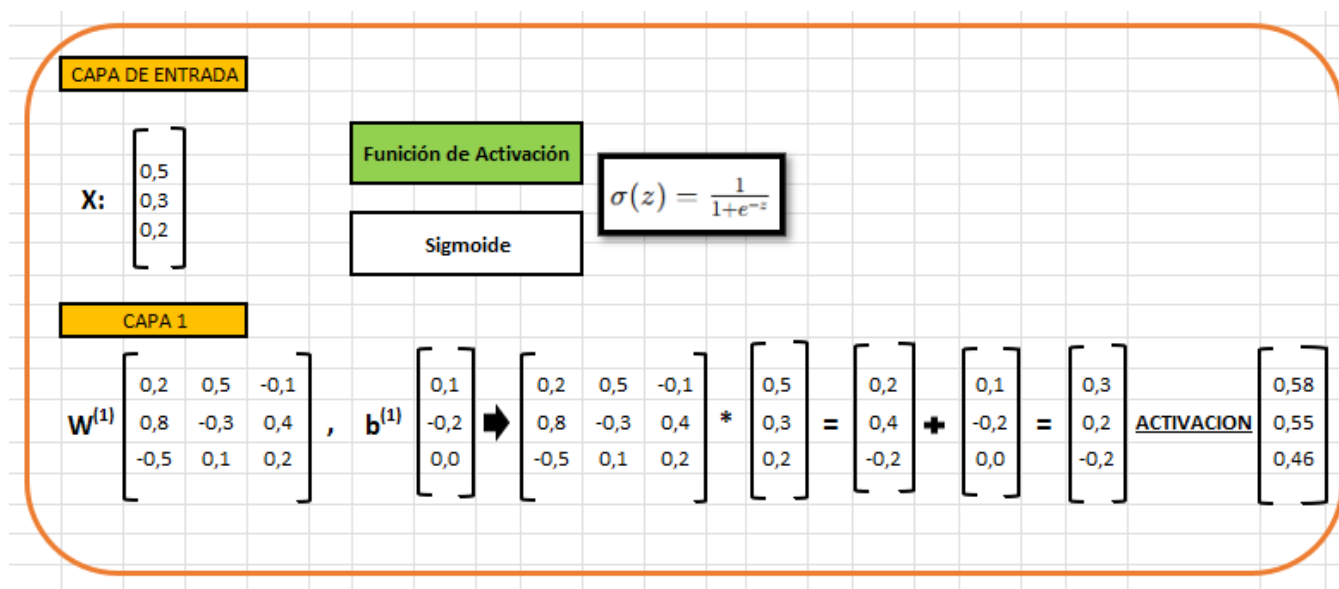
Realice los cálculos paso a paso para determinar a qué categoría pertenece la campaña publicitaria:

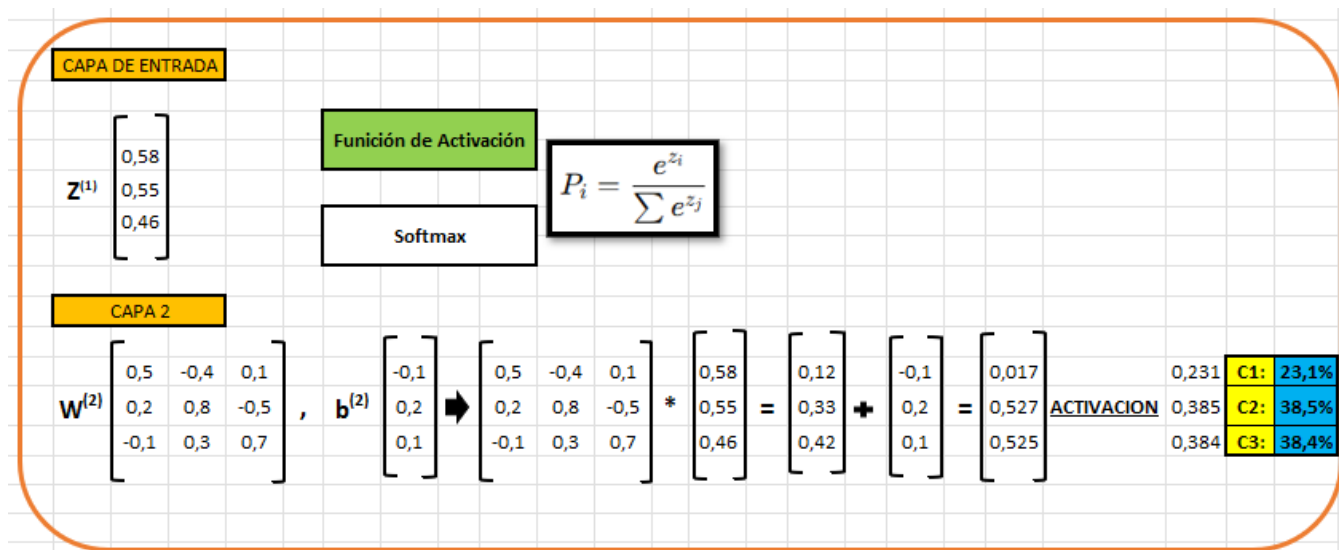
1. **Puntaje de la Capa Oculta ($Z^{(1)}$):** Calcule el producto punto $Z^{(1)} = W^{(1)}X + b^{(1)}$.
2. **Activación de la Capa Oculta ($A^{(1)}$):** Aplique la función sigmoide a cada elemento de $Z^{(1)}$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

3. **Puntaje de la Capa de Salida ($Z^{(2)}$):** Calcule $Z^{(2)} = W^{(2)}A^{(1)} + b^{(2)}$.
4. **Probabilidades de Clase ($A^{(2)}$):** Aplique la función **Softmax** al vector resultante $Z^{(2)}$ para obtener las probabilidades de cada clase (P_1, P_2, P_3):

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}}$$





Preguntas Fase 4

- **Interpretación de Resultados:** Según las probabilidades obtenidas, ¿en qué categoría clasifica la red a esta campaña publicitaria?

RTA: Según las probabilidades que calculamos, la categoría con mayor valor es Impacto Medio (C2), aunque está casi igual que Impacto Alto. Entonces, la red clasifica la campaña como Impacto Medio, porque es la que tiene la probabilidad más alta. En redes neuronales normalmente se elige la clase con el valor mayor, aunque la diferencia sea pequeña.

- **Función de Activación:** ¿Por qué utilizamos *Softmax* en la última capa en lugar de una función lineal o una sigmoide simple? Explique en términos de la suma de las probabilidades.

RTA: Usamos Softmax en la última capa porque convierte los resultados en probabilidades reales, o sea, valores entre 0 y 1 que además suman exactamente 1. Si usáramos una función lineal, los valores podrían ser cualquier número (incluso negativos), y no tendría sentido como probabilidad.

Y si usáramos sigmoide, cada salida sería independiente y no garantizaría que todas las probabilidades sumen 1. En cambio, Softmax sí reparte la “probabilidad total” entre todas las clases, lo que permite comparar fácilmente cuál es la más probable.

- **Rol del Bias:** ¿Que sucedería con la capacidad de representación de la red si todos los vectores de sesgo (*b*) fueran cero?

RTA: Si todos los bias fueran cero, la red perdería flexibilidad. Básicamente, los bias ayudan a ajustar las neuronas, para que no dependan solo de los datos de entrada. Sin bias, la red queda más limitada y le cuesta más aprender patrones complejos. En otras palabras, sería como obligar a que todo pase por el origen, y eso hace que el modelo sea menos preciso.

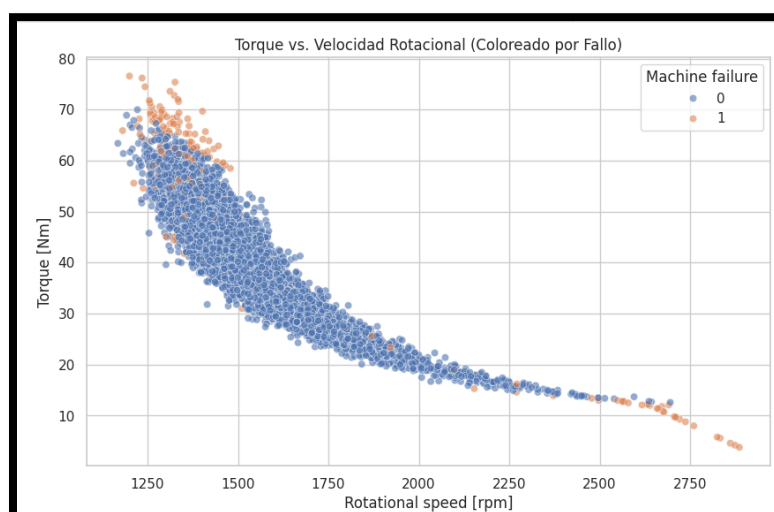
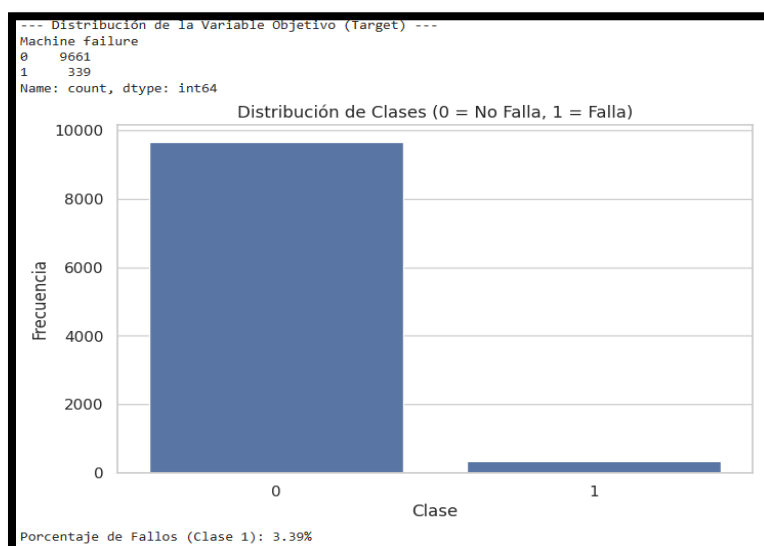
Fase 5: Evaluación de Modelos y Toma de Decisiones (Caso: Mantenimiento Predictivo)

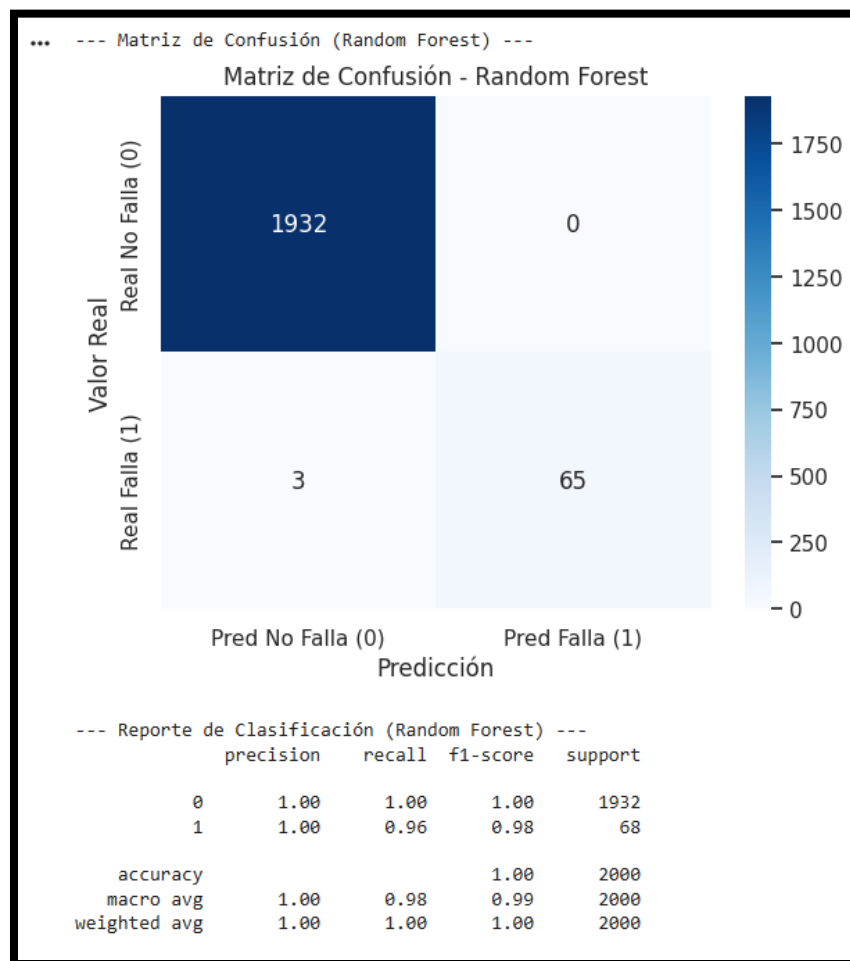
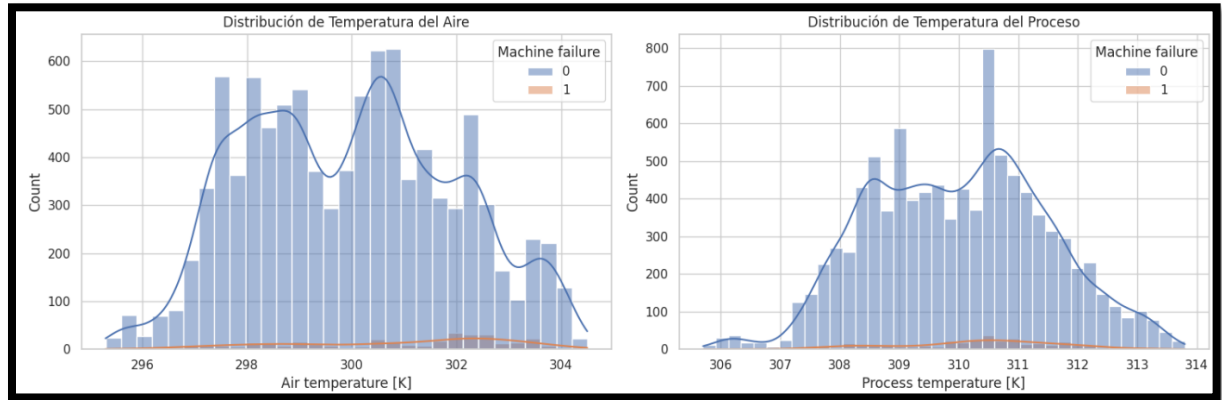
En la manufactura, una parada de maquina no planificada es un fallo critico que genera cuellos de botella y sobrecostos. En esta fase, evaluaremos el desempeño de la Red Neuronal entrenada para el **Mantenimiento Predictivo (PdM)**, donde el objetivo es clasificar si una maquina fallara (Clase 1) o no (Clase 0).

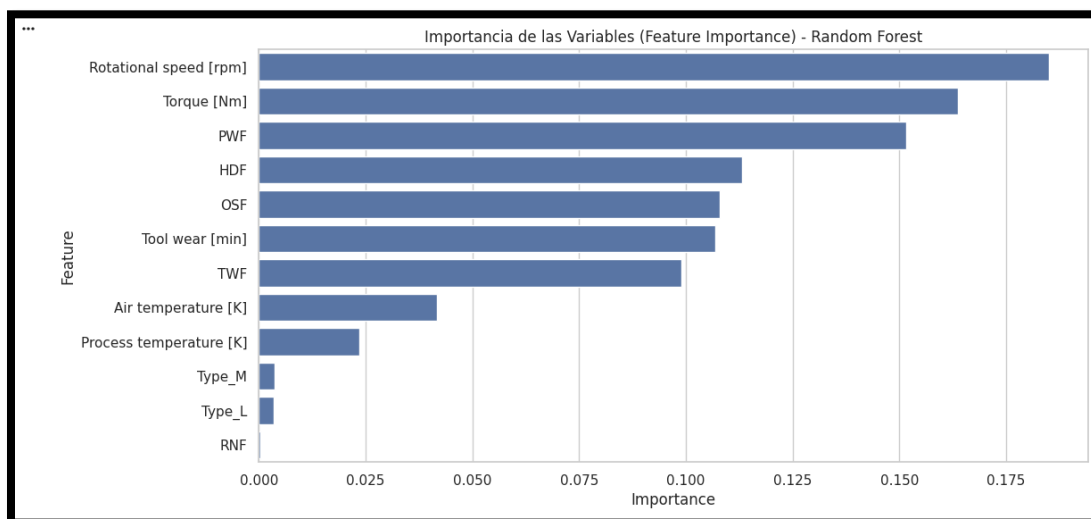
index	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	TWF	HDF	PWF	OSF	RNF
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	0	0	0	0	0
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	0	0	0	0	0
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	0	0	0	0	0
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	0	0	0	0	0
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	0	0	0	0	0

El Desafío del Desbalance de Clases

En plantas reales, las maquinas fallan rara vez (menos del 5 % del tiempo). Esto genera un *data-set* desbalanceado. Observe en el código del notebook como se calculó el diccionario class weights. dict para manejar esta situación.







Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	416
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 16)	528
dropout_1 (Dropout)	(None, 16)	0
dense_2 (Dense)	(None, 1)	17

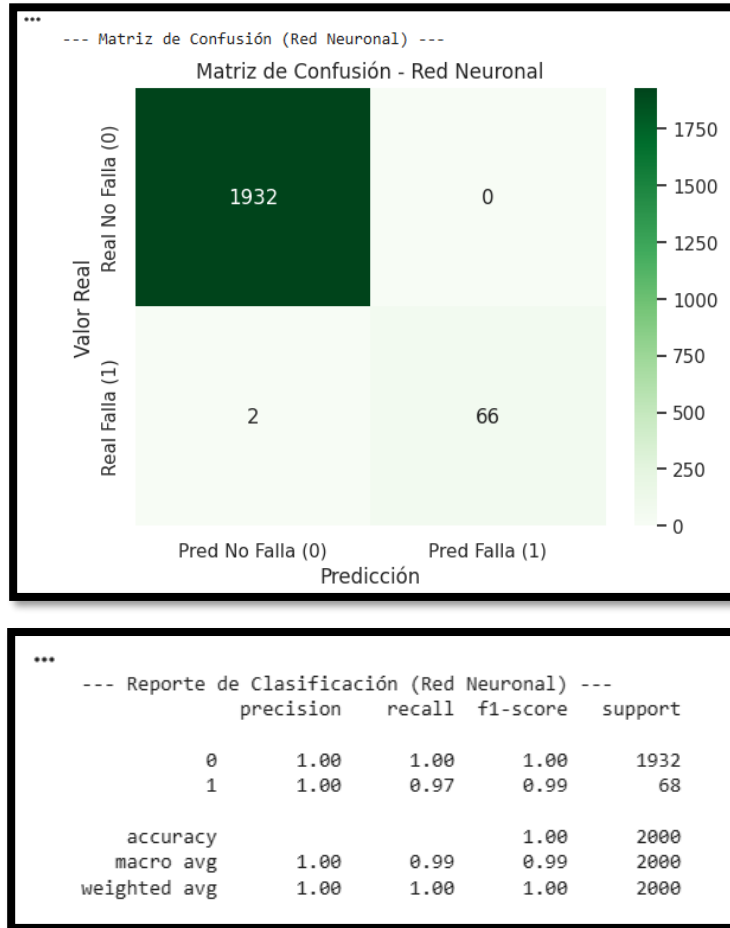
Total params: 961 (3.75 KB)
 Trainable params: 961 (3.75 KB)
 Non-trainable params: 0 (0.00 B)

Preguntas Fase 5A:

- **Análisis de Pesos:** Si el modelo indica que los errores en la Clase 1 (Fallo) pesaran aproximadamente 14.7 veces más que en la Clase 0, ¿que intenta corregir el algoritmo al configurar el parámetro class weight durante el entrenamiento?
- **La Trampa de la Exactitud:** Explique por qué en este caso un *Accuracy* del 95 % podría considerarse un resultado deficiente para la gestión de mantenimiento si el modelo no logra detectar los fallos reales.

Interpretation de la Matriz de Confusion

Utilice la matriz de confusión generada por el *heatmap* de Seaborn en su notebook para completar el siguiente análisis. Suponga que, para su planta de producción, el costo de una **reparación no planificada** (Fallo Real no detectado) es de USD 5,000 y el costo de una **inspección preventiva innecesaria** (Falsa Alarma) es de USD 200.



Tarea de Cálculo de Métricas: A partir de los valores obtenidos en su matriz de confusión (TN, FP, FN, TP), calcule manualmente:

MATRIZ DE CONFUSION		MODELO	
		DEFECTUOSO	BUENO
REAL	DEFECTUOSO	VERDADERO POSITIVO (TP)	FALSO NEGATIVO (FN)
	BUENO	FALSO POSITIVO (FP)	VERDADERO NEGATIVO (TF)

- Precisión (Precisión):** $\frac{TP}{TP+FP}$ — ¿Que tan confiable es el modelo cuando emite una alerta de falla?

PRECISION	$\frac{TP}{TP+FP}$	$\Rightarrow$	$\frac{1932}{1932+2}$	$\equiv$	$\frac{1932}{1934}$	$\equiv$	0,99897	$\equiv$	99,90%
-----------	--------------------	---------------	-----------------------	----------	---------------------	----------	---------	----------	--------

RTA: Esto significa que cuando el modelo predice una falla, prácticamente siempre acierta.

2. **Recall (Sensibilidad):** $\frac{TP}{TP+FN}$ — ¿Qué porcentaje de los fallos reales totales logro capturar el modelo?

RECALL	$\frac{TP}{TP+FN}$	$\Rightarrow$	$\frac{1932}{1932+0}$	$\equiv$	$\frac{1932}{1932}$	$\equiv$	1	$\equiv$	100%
--------	--------------------	---------------	-----------------------	----------	---------------------	----------	---	----------	------

RTA: Esto indica que el modelo logra detectar TODOS los fallos reales, lo cual es excelente para mantenimiento predictivo.

3. **Costo Total de Error:** Calcule el impacto económico mensual del modelo utilizando la formula:

$$\text{Costo} = (FN \times 5000) + (FP \times 200)$$

COSTO TOTAL DE ERROR	$\text{Costo} = (FN \times 5000) + (FP \times 200)$	$\Rightarrow$	$(0 \times 5000) + (2 \times 200)$	$\equiv$	\$ 400,00
----------------------	---	---------------	------------------------------------	----------	-----------

RTA: El impacto económico es de 400.

Preguntas de Reflexión y Negocio Fase 5

- **Trade-off entre Recall y Precisión:** En el contexto de evitar paradas de líneas críticas, ¿preferiría usted un modelo con un *Recall* del 98 % pero una *Precisión* del 60 %, o viceversa? Justifique su respuesta basándose en los costos calculados anteriormente.

RTA:

- **Curva de Aprendizaje:** Observe la gráfica de *loss* (perdida) y *accuracy* durante el entrenamiento. ¿Considera que 30 épocas son suficientes para que el modelo converja? ¿Qué sucede con el error de validación comparado con el de entrenamiento?

RTA: Considero que las 30 épocas sí fueron suficientes, porque el modelo logró buenos resultados y la pérdida de entrenamiento y validación se comportan parecido, significa que el modelo aprendió bien, adicional si fueran muy diferentes, ahí sí habría problema de sobreajuste, pero no parece ser el caso

- **Conclusión del Taller:** Tras recorrer desde la regresión analítica hasta la clasificación con redes neuronales, ¿cómo puede el uso de métricas avanzadas (como el Recall) transformar la rentabilidad de una empresa de manufactura?

RTA: Este taller deja claro que no basta con que un modelo tenga buena exactitud, sino que realmente detecte lo importante, como las fallas también el uso del recall ayuda a evitar pérdidas grandes y tomar mejores decisiones en la empresa. En pocas palabras, estos modelos ayudan a trabajar de forma más inteligente y a reducir riesgos en la producción.

CONCLUSION

En este taller se logró entender que los modelos matemáticos y las redes neuronales tienen una aplicación real en la ingeniería, especialmente para mejorar la toma de decisiones en procesos productivos. A lo largo de las diferentes fases, se pudo ver cómo herramientas como la regresión Ridge, el descenso de gradiente y las redes neuronales permiten analizar problemas de forma más estructurada. Más allá de los cálculos, uno de los aprendizajes más importantes fue la interpretación de los resultados.

No se trata solo de obtener buenos números, sino de entender qué significan y cómo impactan en un contexto real. Por ejemplo, en el caso del mantenimiento predictivo, se evidenció que métricas como el recall son más importantes que la exactitud, ya que permiten detectar fallas y evitar pérdidas económicas.

En general, el taller deja claro que el uso adecuado de estos modelos ayuda a optimizar procesos, reducir riesgos y tomar decisiones más acertadas dentro de una empresa, lo cual es clave en la ingeniería industrial actual.